

Policy Document Intelligence Assistant

Project guide and interview preparation (RAG over organizational policy documents)

1. What the project is

It is an AI assistant that answers questions about company policy documents. A user uploads policy files (PDF, TXT, MD), asks a question in plain English, and gets an answer together with the source file and page it came from. The technique is called **RAG (Retrieval-Augmented Generation)**: instead of letting an AI model answer from memory, the system first *finds* the relevant passages in the documents and then asks the model to answer *only* from those passages. This reduces made-up answers and makes every answer checkable.

Why this problem

Policy documents are long, and people waste time searching them. A normal keyword search misses answers when the wording differs (for example 'vacation' versus 'annual leave'). RAG handles meaning, not just exact words, and citations let the user verify the answer.

2. How it works (end to end)

INGESTION (once per document)
file -> read text (page by page) -> skip if same SHA-256 hash already ingested
-> split into chunks -> turn each chunk into an embedding -> store in vector DB + save metadata

QUESTION (every time)
question -> cache check (same question seen before? return it)
-> search: vector search + BM25 keyword search, merged (hybrid)
-> rerank the candidates -> drop weak matches
-> build prompt (numbered sources, size limit) -> LLM writes the answer
-> attach citations -> save to cache and query log -> return to the UI

- **Chunking:** documents are cut into small pieces, because the search should return a paragraph, not a whole 40-page file.
- **Embedding:** text converted to numbers that capture meaning, so similar meanings end up close together.
- **Hybrid search:** vector search finds meaning matches; BM25 finds exact-term matches (names, numbers, clause words). The two result lists are merged with Reciprocal Rank Fusion (RRF).
- **Reranking:** a second, more careful scoring of the top candidates before they go to the model.
- **Grounded prompt:** the model is told to answer only from the supplied passages and the answer carries [Source: file, Page: N].

3. Architecture and design

The backend follows **Clean Architecture**: code is split into layers, and inner layers never depend on outer ones. This lets me change a technology (for example the vector database or the LLM) without touching the business logic.

Layer (folder)	What lives there	Rule
domain/	Data classes (Chunk, Answer, Citation) and interfaces called ports (Retriever, Reranker, LLMClient, VectorStore ...)	Imports nothing from other layers

Layer (folder)	What lives there	Rule
application/	Use cases: IngestDocuments, AskQuestion; retrieval pipeline; prompt builder; evaluation (recall@k, MRR)	Depends only on domain interfaces
infrastructure/	Real implementations: PDF/text loaders, chunkers, embeddings, ChromaDB, BM25/hybrid retrievers, rerankers, Ollama, SQLite repositories	Implements the domain interfaces
composition/	factories.py and container.py: choose and wire the implementations from settings	The only place that knows concrete classes
main.py	FastAPI endpoints, API-key auth, request logging	Thin: calls use cases

Design patterns used (in simple words)

- **Strategy:** several interchangeable ways to do one job (chunker: fixed or sentence; retriever: dense, bm25, hybrid; reranker: none, keyword, cross_encoder). Chosen with a setting in .env.
- **Factory:** factories.py builds the right object from the setting name and rejects unknown names with a clear error.
- **Dependency Injection:** use cases receive their parts through the constructor, so tests can pass in fakes.
- **Decorator / fallback:** FallbackLLM wraps the main LLM; if Ollama fails, it falls back to an extractive answer built from the retrieved text.
- **Repository:** SQLite access (documents, cache, logs) sits behind interfaces instead of raw SQL inside the endpoints.

4. Tech stack: what, why, and why not the alternative

Technology	Used for	Why this one	Alternative and why not
Python	Whole backend	Best ecosystem for AI/ML libraries	Java/Node: fewer ML libraries; I am strongest in Python
FastAPI + Uvicorn	REST API and the server that runs it	Fast, automatic /docs page, built-in validation with Pydantic, async-ready	Flask: no built-in validation or docs. Django: heavier than needed for an API
Pydantic	Validates requests/responses	Rejects bad input automatically	Manual checks: repetitive and error-prone
PyMuPDF	Reading PDFs page by page	Fast and gives page numbers for citations	PyPDF2: slower and weaker text extraction. OCR tools: only needed for scanned PDFs (not supported yet)
sentence-transformers (all-MiniLM-L6-v2)	Embeddings; optional cross-encoder reranker	Free, runs on CPU, small (about 90 MB), good quality for its size	OpenAI/Azure embeddings: better quality but cost money and send data out. For policy data, local is safer
ChromaDB	Vector database (stores embeddings, finds similar text)	Free, runs locally as files, zero setup	Pinecone/Azure AI Search: managed and scalable but paid. pgvector: good next step for production

Technology	Used for	Why this one	Alternative and why not
BM25 (own implementation)	Keyword search	Exact terms like clause names matter in policies; no extra dependency	A library such as rank_bm25 would also work; writing it keeps it small and testable
Ollama (llama3)	Generates the final answer	Free, runs locally, data never leaves the machine	OpenAI/Azure OpenAI: stronger but paid and external. Easy to add later because LLM is an interface
SQLite	Documents list, answer cache, query log	Built into Python, no server	Postgres: better for many users; would swap behind the repository interfaces
React + Vite + Material UI + Axios	Frontend dashboard	Component-based UI, fast dev server, ready-made components	Plain HTML/JS: more work. Streamlit: faster but less control of the UI
pytest + httpx	Automated tests (23)	Standard, simple; httpx lets the API be tested without a running server	unittest: more boilerplate
Docker + Compose	Packaging and one-command start of API + Ollama	Same behaviour on every machine	Manual setup: error-prone
GitHub Actions	Runs tests on every push	Free for public repos, lives next to the code	Jenkins: needs its own server

5. What was improved: before and now

Area	Before	Now
Code structure	Pipeline logic inside main.py; functions with hard-wired dependencies	Layered architecture with interfaces and dependency injection
Search	Vector search plus a fixed 70/30 keyword blend	Choose dense, BM25 or hybrid (RRF); candidates over-fetched then reranked
Chunking	One method	Fixed-size or sentence-based (never cuts a sentence in half)
Unknown answers	Always tried to answer	Refuses when no passage passes the relevance threshold (threshold is configurable)
LLM failure	Fallback hard-coded inside the LLM function	FallbackLLM wrapper: Ollama first, extractive answer if it fails
Config	Constants and hard-coded CORS list	Settings object from .env, including strategy names and allowed origins
Security / ops	None	API-key auth (constant-time compare), request IDs, latency logging, /health
Tests	6 tests that needed a live server	23 tests that run alone: unit tests with fakes, and in-process API tests
Delivery	Manual install	Dockerfile, docker-compose, GitHub Actions CI
Repo hygiene	Dead code, a stray image, a database file tracked in git	Removed; runtime database ignored

6. Challenges and how they were solved

- **Everything in one file.** Search, generation, caching and SQL were mixed in the endpoints, so nothing could be tested alone. I defined interfaces (ports) first, moved each concern behind one, and wired them in a single container.
- **Merging two kinds of scores.** Vector similarity (0 to 1) and BM25 scores are on different scales, so adding them is unreliable. I used Reciprocal Rank Fusion, which uses each result's *rank*, not its score.
- **Keeping the keyword index current.** BM25 needs an index of all chunks. When new documents are ingested, the index must change too. The retriever rebuilds when the store size changes, and a test covers this.
- **Making tests independent.** The old tests needed a running server and model downloads. With interfaces I could pass in small fake stores, retrievers and LLMs, so the suite runs in about a second.
- **The LLM may be unavailable.** The system should not crash. The fallback wrapper returns an extractive answer from the best sentences with citations.
- **Duplicate uploads.** Re-ingesting the same file would create duplicate chunks and skew results. Each file is hashed with SHA-256 and skipped if already stored.
- **Prompt too long.** Too many chunks overflow the model's context. The prompt builder enforces a character budget and numbers the sources.

7. Honest limitations and next steps

- Retrieval accuracy has not been measured yet. The evaluation tool (recall@k, MRR) exists; I need a labelled question set to produce numbers.
- The relevance threshold defaults to 0.0, so refusal mostly happens when nothing is retrieved. It needs tuning on real questions.
- Single machine: SQLite and local ChromaDB do not scale to many users. Next: Postgres/pgvector.
- Ingestion runs inside the request. Next: background jobs with a status endpoint.
- One shared API key, no per-user access control. Next: JWT/OIDC and per-document permissions.
- No document versioning (an updated policy can leave old chunks), no OCR for scanned files.
- Docker files and the UI against the new backend should be run end to end before demonstrating.

8. Interview questions with natural answers

Use these as a guide and say them in your own words. Only claim what you have actually run or can explain.

A. Introduction and motivation

Q. Tell me about this project.

It is a question-answering system for company policy documents. You upload files, ask a question, and it returns an answer with the file and page it came from. It uses RAG: it first searches the documents for relevant passages and then has a language model answer only from those. I started from a basic prototype and rebuilt the backend into a clean, tested, containerized system with hybrid search.

Q. Why did you choose policy documents?

Policies are long and people struggle to find specific rules. They also need accuracy and traceability, which makes them a good fit for RAG, because every answer can point to its source.

Q. What was your own contribution?

The frontend and the basic pipeline already existed. I redesigned the backend into layers, added BM25 and hybrid retrieval with reranking, the refusal behaviour, the LLM fallback, API-key auth and logging, and I wrote the tests, Docker setup and CI.

B. RAG concepts

Q. What is RAG and why not just ask the LLM directly?

A plain LLM answers from its training data, so it can be outdated or invent details, and it has never seen my company's documents. RAG gives it the right passages at question time, so the answer is grounded and can be cited.

Q. What is an embedding?

A list of numbers that represents the meaning of a text. Texts with similar meaning get similar numbers, so I can find relevant chunks by comparing numbers instead of matching words.

Q. Why chunk the documents? How did you decide the size?

Searching whole documents is too coarse and models have a limited context. I use about 500 characters with 50 overlap by default, and a sentence-based chunker that never cuts a sentence. These values are configurable. I would tune them with the evaluation tool rather than guess.

Q. Why hybrid search instead of only vector search?

Vector search understands meaning but can miss exact terms such as a clause name or a number. BM25 is the opposite. Combining both covers both cases, which matters in policy text.

Q. How do you combine the two result lists?

With Reciprocal Rank Fusion. Their scores are on different scales, so instead of adding scores I combine the ranks. A chunk that both searches rank highly rises to the top.

Q. What does the reranker do?

The first search is fast but rough, so I fetch more candidates than needed, then re-score them more carefully and keep the top few. The default reranker uses keyword overlap; an optional cross-encoder model gives better quality at higher cost.

Q. How do you reduce hallucination?

Three ways: the prompt tells the model to answer only from the supplied passages; every answer carries citations so a person can check; and there is a relevance threshold so the system can say it found nothing instead of guessing. The threshold still needs tuning, so I describe it as reducing hallucination, not eliminating it.

Q. How would you measure whether it works?

I built an evaluation module that computes recall@k (is the right page in the top k results) and MRR (how high the first right result ranks) over a labelled question set. I would compare dense, BM25 and hybrid, with and without reranking, and keep the best setting.

C. Design and code quality

Q. What is Clean Architecture and why did you use it?

Code is split into layers where inner layers do not know about outer ones. The business logic depends on interfaces, not on ChromaDB or Ollama directly. So I can change the vector database or the model by writing one adapter class, and I can test the logic with fakes.

Q. Give an example of the Strategy pattern in your code.

The retriever. Dense, BM25 and hybrid all implement the same interface. A setting in .env picks one, and the rest of the code does not change. Adding a new one is one class and one line in the factory.

Q. What is dependency injection here?

Classes receive the parts they need through their constructor instead of creating them. A container builds the real parts at startup; in tests I pass fake ones.

Q. How is the code tested?

23 pytest tests. Unit tests use small fake stores, retrievers and LLMs to check chunking, BM25, RRF, reranking, the fallback, caching, refusal and the evaluation metrics. API tests use FastAPI's test client with a stubbed container, so they need no server or models.

D. Technology choices

Q. Why FastAPI?

It is quick to build with, validates input through Pydantic, and generates interactive docs automatically. It also works well with async code if I need it later.

Q. Why ChromaDB, and what would you use in production?

It is free and needs no server, which suits a local project. In production I would move to something like pgvector or a managed service for scaling, backups and access control. Because the vector store is behind an interface, that is a contained change.

Q. Why a local model (Ollama) instead of the OpenAI API?

Policy documents can be sensitive, so keeping everything local avoids sending data outside, and it is free. The trade-off is lower answer quality and the need for enough RAM. Because the LLM is an interface, I could add Azure OpenAI as another implementation.

Q. Why not deploy it on Azure or AWS?

It was not needed to build and demonstrate the system, and it would add cost and setup that make it harder for others to run. I designed it so the cloud move is small: the model, vector store and database are swappable, and the Docker image can run on a container service.

Q. How do the frontend and backend talk?

The React app calls the FastAPI backend over HTTP using Axios, for example POST /ask. The backend address comes from a setting, and the backend allows the frontend's origin through CORS.

E. Docker, security, operations

Q. Explain your Docker setup.

The Dockerfile starts from a slim Python image, installs the requirements first so that layer is cached when only code changes, copies the backend, and runs as a non-root user with a health check. Docker Compose starts the API together with Ollama and uses volumes so the databases and downloaded models survive restarts. A .dockerignore keeps secrets and junk out of the image. I would verify the full build before presenting it.

Q. How do you handle secrets and security?

Settings come from a .env file that is not committed or copied into the image. Data endpoints can require an API key, compared in constant time. It is basic: for real use I would add user authentication, per-document access control and rate limiting.

Q. How would you scale this?

Move metadata and vectors to Postgres with pgvector, run ingestion as background jobs, run several API instances behind a load balancer, cache embeddings and answers in a shared store like Redis, and add metrics and tracing.

Q. What happens when a policy document is updated?

Today a changed file has a new hash, so it is ingested as a new document and old chunks may remain. That is a known gap. I would add versioning so the new version replaces the old one.

F. Difficulties and reflection

Q. What was the hardest part?

Restructuring the existing code without breaking behaviour. Search, generation, caching and database code were mixed together. I defined the interfaces first, moved one concern at a time, and kept the tests running so I could see that nothing broke.

Q. What problem did you hit with hybrid search?

The two search methods return scores on different scales, so simply adding them is misleading. Switching to rank-based fusion fixed it. I also had to make sure the keyword index refreshed when new documents were added, and I wrote a test for that.

Q. What would you do differently?

Build the labelled evaluation set at the start, so every design choice, such as chunk size, retriever and reranker, is backed by numbers instead of intuition.

Q. What are the weaknesses of the project?

Accuracy is not measured yet, the refusal threshold is not tuned, it is single-machine, and there is no document versioning or per-user permissions. I know the fixes and have listed them as next steps.

Q. What did you learn?

That retrieval quality matters more than the model, that interfaces make a system easier to change and test, and that you need measurements, not impressions, to claim something works.

9. 60-second pitch

"I built a question-answering assistant for company policy documents. It uses RAG: it searches the documents with a mix of vector and keyword search, re-ranks the results, and has a local language model write an answer that always cites the file and page. I rebuilt the backend using clean architecture, so the search method, reranker and model can be changed from a settings file. I added API-key security, logging, 23 automated tests, and Docker with a CI pipeline. The next step I am working on is measuring retrieval accuracy with a labelled question set."

10. Resume entry

Policy Document Q&A Assistant (RAG) | Python, FastAPI, ChromaDB, React, Docker

- Built an AI assistant that answers questions from policy documents and cites the source file and page for every answer.
- Implemented hybrid retrieval (vector + BM25 with rank fusion) and a reranking step; the system declines to answer when no relevant passage is found.
- Redesigned the backend with Clean Architecture, Factory and Strategy patterns so the retriever, reranker and LLM are switched through configuration.
- Added API-key authentication, request logging, 23 automated tests, Docker Compose setup and GitHub Actions CI.